

P0 Suggestions

Implementation Language and AI

As per the syllabus, the project must be in C/C++. Other languages require approval. In either case, you must match the requirements, including invocation. Structural and coding standards must be satisfied, appropriate to the language.

Even if you use C, I would still suggest using the C++ string class as safer and using STL container instead of arrays.

Beyond that, use what you are comfortable with or can learn/review as needed.

You can use AI to help with your code, but you must "own" the code at the end, that is, you must be able to explain how it works and make small changes as potentially needed for changing requirements. It is likely difficult, if possible, to generate the project using a single prompt. But it is quite possible to generate solutions for aspects of the project and then combine them. If you generate the entire project, show it to me, please.

Development and Deployment Environment

You can develop on any environment you are comfortable with but the submission must be on Delmar, details on the [Syllabus](#) page.

If you develop elsewhere, it is imperative to transfer properly and retest in the Delmar environment. Make sure to provide the right makefile using the proper compiler etc.

The Task

Using top-down decomposition you have 3 tasks in main:

1. *Process command arguments*, set up data to work regardless of source, check if file is readable, set the basename for the output file, etc.
2. *Build the tree*
3. *Traverse the tree* three different ways, generating outputs

The main function should handle the 3 functionalities. #1 should be handled inside of main or in a static helper implemented in the same file as main, functions for #2 and #3 should be in separate source(s) . Any node types should be defined in a separate header file.

The tree is built using the frequency of a given string in the file. Because the frequency is unknown when seeing a string, the program should make two passes over the input. The first pass will collect

the strings along with frequencies (use a container, a dictionary, etc.). The second pass will read the container and construct the tree without rearranging.

For development/testing purposes, 2 must be done before 3. 1 can be done before 2/3 or after. Do not mix up working on 1 and 2/3. I would suggest working out 1 first and testing so it can be reused later, and then working on 2 and 3. In working on part 1, I would suggest first implement just the argument and then add the option to not have the argument. Use source control.

Invocation argument, step 1 above

Processing either keyboard or file input can be done one of these two ways:

1. If keyboard input ("P0" or "P0 < file", the program doesn't know which but the file there was no argument), read the input into a temporary file, close and reopen to read
Otherwise you have the file, just append the extension and open it to read
Afterwards the rest of the program always processes file input
2. If keyboard input, set file pointer to stdin otherwise set file pointer to the actual file (after adding extension and opening), then process always from the file pointer

Structure

Sample structure for main.c

```
#include "node.h"
#include "tree.h"
int main(int argc, char* argv[]) {
    // process command line arguments first and error if improper arguments
    // if filename given, make sure file is readable, error otherwise
    // set file pointers or read to temporary file so that below this point there is only one version of
    the code
    // this could be through a call to a helper function

    node_t *root = buildTree(file);
    printPreorder(root);
    printInorder(root);
    printPostorder(root);
    return 0;
}
```

Printing in Traversals

Ideas for printing tree with indentations using printf here

```
static void printPreorder(nodeType *rootP, int level) {
    if (rootP == NULL) return;
    printf("%*c%d:%-9s ", level*2, ' ', level, NodeId.info); // assume some info printed as string
    printf("\n");
    printPreorder(rootP->left, level+1);
    printPreorder(rootP->right, level+1);
}
```